

# Semántica de MINIJAVA – Parte I

## Declaraciones

Compiladores e Intérpretes – 2026

### 1 Alcance

Este documento define los controles semánticos de declaraciones de la Etapa 3. En esta etapa se construye y consolida la tabla de símbolos. No se chequean los cuerpos de métodos ni constructores. Esos controles corresponden a la Etapa 4.

Las reglas se aplican sobre programas sintácticamente válidos según las *Reglas de Sintaxis de MiniJava 2026*.

#### Regla de oro

Todo control o comportamiento que no esté especificado en este documento sigue las reglas de Java.

### 2 Tabla de símbolos

El compilador debe mantener una tabla global de tipos. Esa tabla contiene las clases e interfaces declaradas por el programa y las entidades predefinidas.

Cada clase debe registrar: nombre, parámetro genérico opcional, tipo heredado o interfaz implementada, atributos, métodos y constructores.

Cada interfaz debe registrar: nombre, parámetro genérico opcional, interfaz extendida y métodos.

Cada método debe registrar: nombre, condición de estático, tipo de retorno y parámetros ordenados. Cada constructor debe registrar sus parámetros ordenados. Cada parámetro debe registrar nombre, tipo y posición. Todas las entidades deben conservar el token de su declaración.

La identidad de un método o constructor incluye su aridad, es decir, la cantidad de parámetros formales.

### 3 Tipos

Los tipos son `boolean`, `char`, `int`, tipos referencia, parámetros de tipo y arreglos. `void` sólo se utiliza como retorno de un método.

Un arreglo puede tener cualquier cantidad de dimensiones. Su tipo base puede ser primitivo, referencia o parámetro de tipo.

Un tipo referencia debe nombrar una clase o interfaz existente, o una entidad predefinida. Puede tener un argumento genérico opcional: `Caja`, `Caja<T>` o `Caja<String>`.

Si un tipo referencia tiene argumento genérico, el tipo referenciado debe declarar un parámetro genérico. El argumento debe ser un parámetro de tipo válido en el contexto actual o el nombre de una clase o interfaz existente.

Un parámetro de tipo sólo es válido dentro de la clase o interfaz que lo declara. Por ejemplo, `T` es válido en los atributos y encabezados de `Caja`:

```
class Caja<T> {
    T contenido;
    T obtener() { }
}
```

## 4 Clases e interfaces

Clases e interfaces comparten la tabla global de tipos. Cada nombre de tipo puede tener una sola declaración. Tampoco pueden redeclararse `Object`, `String` ni `System`.

Una clase con `extends R` debe extender una clase existente `R`. Una clase con `implements R` debe implementar una interfaz existente `R`. Una clase sin relación explícita extiende `Object`.

Una interfaz con `extends R` debe extender una interfaz existente `R`.

Las cadenas de herencia entre clases y de extensión entre interfaces deben ser acíclicas.

## 5 Genericidad y herencia

Una clase o interfaz declara, como máximo, un parámetro de tipo. Ese parámetro puede utilizarse en los tipos de sus atributos, retornos y parámetros formales.

Al heredar de una clase o interfaz genérica con argumento explícito, los tipos paramétricos de los miembros heredados se instancian con ese argumento.

Al heredar de una clase o interfaz genérica sin argumento explícito, se conserva el nombre del parámetro de tipo del tipo heredado. La consolidación no lo elimina ni lo reemplaza por un tipo concreto.

```
class A<T> {
    T obtener() { }
}
class B extends A { }
```

Luego de consolidar `B`, el método heredado `obtener` conserva retorno `T`. Esta regla también se aplica a `implements` y a la extensión de interfaces.

## 6 Miembros

### 6.1 Atributos

Un atributo debe tener un tipo válido. Una clase no puede declarar dos atributos con el mismo nombre. Una clase tampoco puede declarar un atributo con el mismo nombre que un atributo heredado.

### 6.2 Métodos

Un método debe tener tipo de retorno válido y parámetros de tipos válidos.

MiniJava permite sobrecarga simple. Dos métodos de una misma clase o interfaz pueden tener el mismo nombre si poseen distinta aridad. La clave de un método es (*nombre, aridad*).

Dos métodos de una misma clase o interfaz con igual nombre e igual aridad son una declaración repetida, aunque difieran en los tipos o nombres de sus parámetros.

```
void mostrar() { }
void mostrar(int n) { }           // válido: mostrar/0 y mostrar/1

void imprimir(int n) { }
void imprimir(char c) { }        // error: imprimir/1 repetido
```

Si un método de instancia tiene el mismo nombre y aridad que un método de instancia heredado, lo redefine. Ambas firmas deben ser exactamente iguales: misma cantidad de parámetros, mismos tipos de parámetros en el mismo orden y mismo tipo de retorno. Los nombres de parámetros no forman parte de la firma. La comparación se realiza luego de aplicar la instanciación genérica que corresponda.

Los métodos estáticos no participan de la redefinición. Si un ancestro declara un método estático con clave (*nombre, aridad*), ninguna descendiente puede declarar otro método con esa clave, sea estático o de instancia. Tampoco puede declararse un método estático con la misma clave que un método de instancia heredado.

Una clase que implementa una interfaz debe poseer, directa o hereditariamente, un método de instancia compatible para cada método de la interfaz. Un método estático no implementa un método de interfaz.

### 6.3 Constructores

Un constructor debe tener el mismo nombre que su clase. Una clase puede tener varios constructores con distinta aridad. Dos constructores de una misma clase con igual aridad son una declaración repetida, aunque difieran en los tipos de sus parámetros.

Los parámetros de un constructor deben tener tipos válidos y nombres no repetidos.

Si una clase no declara constructores, el compilador debe agregar un constructor por defecto sin parámetros y con cuerpo vacío. Si la clase declara al menos un constructor, no se agrega el constructor por defecto.

### 6.4 Parámetros formales

Los parámetros de un método o constructor están ordenados. En una misma unidad no pueden repetirse nombres de parámetros. Cada parámetro debe tener un tipo válido.

## 7 Entidades predefinidas

Antes de analizar el programa, la tabla global debe contener `Object`, `String` y `System`.

- `Object` declara:

```
static void debugPrint(int i)
String toString()
```

- `String` es el tipo de los literales de cadena, hereda de `Object`, no tiene métodos propios
- `System` hereda de `Object`, declara:

```
static int read()
static void printB(boolean b)
static void printI(int i)
static void println()
static void printBln(boolean b)
static void printIln(int i)
static void printC(char c)
static void printS(String s)
static void printCln(char c)
static void printSln(String s)
```

## 8 Procesamiento

En una primera pasada se construyen las tablas de símbolos. En esta pasada se controlan los nombres repetidos de tipos, atributos, métodos con igual nombre y aridad, constructores con igual aridad y parámetros de una misma unidad.

En una segunda pasada se controlan tipos válidos, parámetros genéricos, relaciones entre clases e interfaces, circularidad, redefiniciones y contratos de interfaces.

Finalmente se consolidan las clases e interfaces. La consolidación incorpora los miembros heredados, respeta las redefiniciones válidas e instancia o conserva los parámetros de tipo según la relación de herencia.